Compte-Rendu de Travaux Pratiques : Blockchain

Matière : Blockchain & Décentralisation **TP 2 :** Création d'une blockchain simple en Python **Étudiant :** Ahmed Channoufi

Exercice 1 : Découverte de la fonction de hachage SHA-256

Objectif : Expérimenter la fonction SHA-256 avec la bibliothèque standard Python `hashlib`, observer l'effet d'avalanche et la taille fixe de l'empreinte.

1. Code Python

```
import hashlib

texte = "Blockchain"
hash_texte = hashlib.sha256(texte.encode("utf-8")).hexdigest()

texte2 = "blockchain"
hash2 = hashlib.sha256(texte2.encode("utf-8")).hexdigest()

texte3 = "Blockchain et cryptographie"
hash3 = hashlib.sha256(texte3.encode("utf-8")).hexdigest()
```

2. Sortie dans le terminal

```
Texte : Blockchain
Hash  : 625da44e4eaf58d61cf048d168aa6f5e492dea166d8bb54ec06c30de07db57e1

Texte : blockchain
Hash  : ef7797e13d3a75526946a3bcf00daec9fc9c9c4d51ddc7cc5df888f74dd434d1

Texte : Blockchain et cryptographie
Hash  : f8fc99c0038743eb1434318e40676c5097187dea6147bbfcb4a67f3ec92ab18d
```

3. Réponses aux questions

Q1. Deux textes différents donnent-ils le même hash ?
Non. "Blockchain" et "blockchain" produisent des empreintes totalement différentes. Une seule lettre modifiée transforme l'intégralité du hash (effet avalanche).

Q2. Longueur du hash SHA-256 en caractères hexadécimaux ?
Exactement 64 caractères hexadécimaux (256 bits / 32 octets).

Q3. La taille du texte d'entrée influence-t-elle la taille du hash ?
Non, la sortie fait toujours 64 caractères quelle que soit la longueur du texte initial.

Q4. Utilité du hachage dans la blockchain ?
Il génère une empreinte numérique unique pour chaque bloc. Toute altération est immédiatement détectable.

Q5. Peut-on inverser le hash pour retrouver le texte ?
Non, SHA-256 est une fonction à sens unique mathématiquement irréversible.

Exercice 2 : Création de la classe Bloc

Objectif : Concevoir la structure de données d'un bloc avec `index`, `transaction`, `previous_hash` et `hash`.

1. Code Python

```
class Bloc:
    def __init__(self, index, transaction, previous_hash):
```

```
        self.index = index
        self.transaction = transaction
        self.previous_hash = previous_hash
        self.hash = self.calculer_hash()

    def calculer_hash(self):
        contenu = str(self.index) + "|" + self.transaction + "|" + self.previous_hash
        return hashlib.sha256(contenu.encode("utf-8")).hexdigest()

bloc1 = Bloc(1, "Alice envoie 10 euros a Bob", "0")
bloc1_modifie = Bloc(1, "Alice envoie 100 euros a Bob", "0")
```

2. Sortie dans le terminal

```
Bloc 1 initial  : 0235e2abe7e4bf2794746aabba340a82de46da1f07b64ec08e7bd175ce1d1c85
Bloc 1 modifie  : dbb5eaea6cf6adeadf26a169376384c15b916826084da502ad627f503370a76c
Les deux hash sont identiques ? False
```

3. Réponses aux questions

Q1. Rôle de __init__() ?
Constructeur de l'objet : initialise les attributs du bloc lors de l'instanciation.

Q2. Rôle de self.hash = self.calculer_hash() ?
Calcule et enregistre immédiatement le hash du bloc à sa création.

Q3. Pourquoi str(self.index) ?
Convertit le nombre entier en chaîne de caractères pour la concaténation et l'encodage utf-8.

Q4. Attributs utilisés pour le hash ?
L'index, la transaction et le previous_hash séparés par des délimiteurs.

Q5. Constat après modification ?
Le hash change complètement, prouvant la sensibilité du bloc aux altérations.

Exercice 3 : Chaînage de trois blocs

Objectif : Relier cryptographiquement plusieurs blocs successifs via le champ `previous_hash`.

1. Code Python

```
bloc1 = Bloc(1, "Alice envoie 10 euros a Bob", "0")
bloc2 = Bloc(2, "Bob envoie 5 euros a Charlie", bloc1.hash)
bloc3 = Bloc(3, "Charlie envoie 2 euros a Alice", bloc2.hash)

blockchain = [bloc1, bloc2, bloc3]
```

2. Sortie dans le terminal

```
Bloc 1 | Prev: 0                                | Hash: 0235e2abe7e4bf27...
Bloc 2 | Prev: 0235e2abe7e4bf2794746aabba340... | Hash: 1e4e604e0e5727b7...
Bloc 3 | Prev: 1e4e604e0e5727b738a61b5000b09... | Hash: 4387302c6a3f17e6...

Verification du lien Bloc 1 -> Bloc 2 : True
Verification du lien Bloc 2 -> Bloc 3 : True
```

3. Réponses aux questions

Q1. Comment les blocs sont-ils reliés ?
Chaque bloc intègre dans son attribut `previous_hash` l'empreinte du bloc précédent.

Q2. Pourquoi Bloc 2 utilise bloc1.hash ?
Pour ancrer le deuxième bloc à l'état précis du premier.

Q3. Rôle de la liste blockchain ?
Elle maintient l'ordre chronologique des blocs en mémoire.

Q4. Si on remplace bloc1.hash par '12345' ?
La liaison est brisée car `previous_hash` ne correspond plus au hash réel du bloc 1.

Exercice 4 : Vérification de l'intégrité de la blockchain

Objectif : Développer un algorithme de vérification qui valide à la fois l'intégrité du contenu et la validité des liens.

1. Code Python

```
def verifier_blockchain(chaine):
    for i in range(len(chaine)):
        bloc = chaine[i]
        # 1. Verification du contenu interne
        if bloc.hash != bloc.calculer_hash():
            return False
        # 2. Verification du chainage
        if i > 0 and bloc.previous_hash != chaine[i - 1].hash:
            return False
    return True

# Test normal puis test avec falsification
print("Chaine valide :", verifier_blockchain(blockchain))
bloc1.transaction = "Alice envoie 1000 euros a Bob"
print("Chaine apres modif :", verifier_blockchain(blockchain))
```

2. Sortie dans le terminal

```
Chaine valide : True
[ERREUR] Bloc 1 : le contenu a ete modifie !
Chaine apres modif : False
```

3. Réponses aux questions

> **Q1. Pourquoi recalculer le hash lors de la vérification ?**
> Pour vérifier que les données actuelles en mémoire correspondent bien au hash enregistré à l'origine.
>
> **Q2. Pourquoi vérifier previous_hash ?**
> Pour s'assurer qu'aucun bloc intermédiaire n'a été inséré, retiré ou altéré.
>
> **Q3. Cette fonction suffit-elle pour sécuriser une blockchain en production ?**
> Non. Sur une seule machine, un attaquant peut recalculer tous les hashs d'un coup. En production, il faut la décentralisation (nœuds P2P), les signatures numériques et un consensus (PoW / PoS).

Exercice 5 : Ajout d'un 4ème bloc

Objectif : Étendre la chaîne avec une nouvelle transaction et confirmer la validité globale.

1. Code Python

```
bloc4 = Bloc(4, "David envoie 15 euros a Alice", bloc3.hash)
blockchain = [bloc1, bloc2, bloc3, bloc4]
valide = verifier_blockchain(blockchain)
print("Blockchain valide :", valide)
```

2. Sortie dans le terminal

```
Bloc 4 | David envoie 15 euros a Alice | Prev: 4387302c6a3f17e6... | Hash: 271d8970d7452188...
Verification de la chaine complete : True
```

3. Réponses aux questions

Q1. Quel hash doit utiliser le Bloc 4 ?
Obligatoirement le hash du Bloc 3 (`bloc3.hash`).

Q2. Pourquoi ne pas utiliser le hash du Bloc 1 ?
La chaîne est linéaire et séquentielle. Chaque bloc doit pointer vers son prédécesseur immédiat.

Exercice 6 : Expérimentations de falsification & Compréhension

Objectif : Tester la robustesse de la chaîne face à 5 tentatives de falsification et analyser les résultats.

1. Tableau récapitulatif des 5 manipulations

N°	Manipulation testée	Résultat	Explication
1	Modifier transaction du bloc 2 sans recalculer son hash	**False**	Le hash stocké ne correspond plus au texte modifié.
2	Modifier previous_hash du bloc 3	**False**	Le lien cryptographique avec le bloc 2 est rompu.
3	Ajouter proprement un 5ème bloc	**True**	Le bloc 5 référence correctement le hash du bloc 4.
4	Modifier le bloc 1 et recalculer uniquement son hash	**False**	Le bloc 2 référence encore l'ancien hash du bloc 1.
5	Modifier le bloc 1 et recalculer TOUS les hashs suivants	**True (local)**	L'intégrité locale est rétablie, mais sur un vrai réseau les pairs rejetteront cette version.

2. Réponses aux questions de compréhension (Partie IV)

Q1. Qu'est-ce qu'une fonction de hachage cryptographique ?
Une fonction mathématique déterministe qui convertit des données quelconques en une empreinte de taille fixe, impossible à inverser.

Q2. Différence entre la transaction et le hash ?
La transaction est la donnée métier (le transfert de fonds). Le hash est l'empreinte de sécurité calculée à partir de celle-ci.

Q3. Qu'est-ce que le bloc Genesis ?
Le premier bloc de la chaîne, créé sans bloc antérieur (`previous_hash = "0"`).

Q4. Pourquoi un attaquant ne peut pas tricher sur un vrai réseau ?

Même s'il recalcule tous les hashs sur sa machine (manip 5), les autres nœuds du réseau possèdent la chaîne originale majoritaire et rejetteront sa version.

Q5. Que manque-t-il à notre script pour être une blockchain réelle ?

Un protocole réseau pair-à-pair (P2P), des signatures cryptographiques (clés publiques/privées) et un mécanisme de consensus décentralisé (Proof of Work / Proof of Stake).